Gradient Descent: Theory, Mathematics, and Optimization

A Comprehensive Guide to Optimization Algorithms

Exploring the mathematical foundations, variants, and practical applications of gradient descent in machine learning and deep learning.

Autor: Maru

February 2026

1. Introduction to Gradient Descent

Gradient descent is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function. The core idea is simple yet powerful: repeatedly take steps in the opposite direction of the gradient (the direction of the steepest ascent) of the function at the current point, because this is the direction in which the function decreases most rapidly.

In machine learning, gradient descent is the workhorse algorithm behind training neural networks, linear regression, logistic regression, and many other models. The algorithm minimizes a **cost function** (or loss function) by updating parameters iteratively.

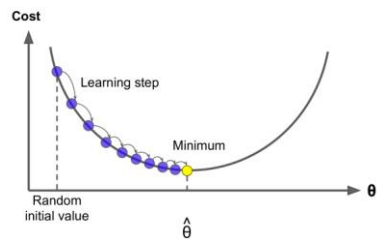

Figure 1: The gradient points in the direction of steepest ascent. Taking steps in the opposite direction minimizes the cost function.

2. Mathematical Foundations

2.1 The Concept of Derivatives

At the heart of gradient descent lies the concept of the **derivative**. The derivative of a function at a point represents the instantaneous rate of change or the slope of the tangent line at that point.

For a function f(x), the derivative f'(x) is defined as the limit of the difference quotient as the interval approaches zero:

$$\frac{d}{dx}f(x) = \lim_{h\to 0}\frac{f(x+h)-f(x)}{h}$$

Definition of the derivative as a limit

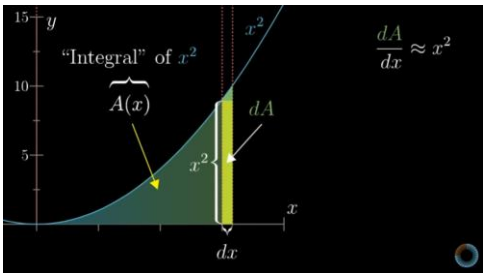

Figure 2: Geometric interpretation of the derivative as the slope of the secant line approaching the tangent

2.2 Partial Derivatives and Gradients

When dealing with multivariate functions $f(x_1, x_2, ..., x_n)$, we use **partial derivatives** to measure the rate of change with respect to each variable while holding others constant. The **gradient** is the vector of all partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

The gradient vector of a multivariate function

The gradient points in the direction of the steepest increase of the function, with its magnitude representing the rate of increase in that direction.

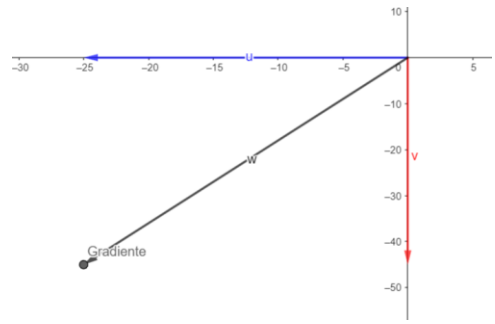

Figure 3: 3D visualization of a multivariate function z = f(x,y) and its domain

3. The Gradient Descent Algorithm

3.1 The Update Rule

The fundamental update rule for gradient descent is straightforward. At each iteration, parameters are updated by moving in the opposite direction of the gradient, scaled by a learning rate α:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha\nabla J(\boldsymbol{\theta}_t)$$

Gradient descent parameter update rule

Where:

- θ represents the parameters (weights) being optimized
- α (alpha) is the **learning rate** - a hyperparameter controlling step size
- ∇J(θ) is the gradient of the cost function J with respect to θ
- t denotes the iteration number

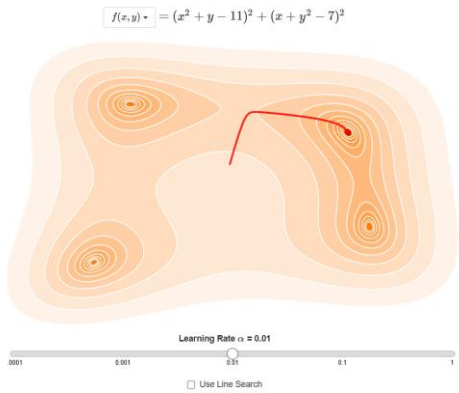

Figure 4: Gradient descent converging to the minimum of a convex cost function

3.2 The Cost Function

For linear regression, the typical cost function is the Mean Squared Error (MSE). For a neural network, this might be cross-entropy loss. The general form is:

$$J(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} L(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

General cost function as the average of loss over m training examples

Where L is the loss function for a single example, h is the hypothesis function, and m is the number of training examples.

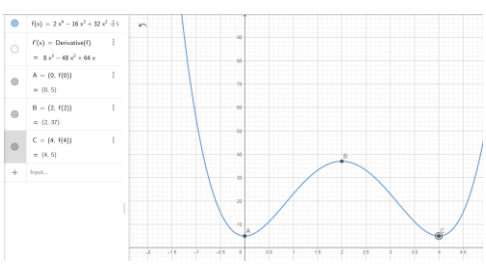

Figure 5: Cost function behavior and critical points analysis using the first derivative criterion

4. Types of Gradient Descent

There are three main variants of gradient descent, each differing in the amount of data used to compute the gradient:

Table 1: Comparison of gradient descent variants

Variant	Batch Size	Speed	Memory	Precision
Batch GD	Entire dataset	Slow per epoch	High	Very precise
Stochastic GD	1 sample	Fast per epoch	Low	Noisy updates
Mini-batch GD	n samples (e.g., 32, 64)	Balanced	Moderate	Balance of speed and stability

4.1 Batch Gradient Descent

Batch Gradient Descent computes the gradient using the entire dataset. This provides a stable, accurate estimate of the gradient but is computationally expensive for large datasets.

$$\nabla J(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

Batch gradient uses all m training examples

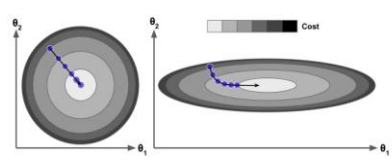

Figure 6: Batch gradient descent showing smooth convergence in a 2D parameter space

4.2 Stochastic Gradient Descent (SGD)

SGD updates parameters using only a single training example at a time. This makes it faster and enables online learning, but the updates are noisy and have high variance.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

SGD update using a single randomly selected example

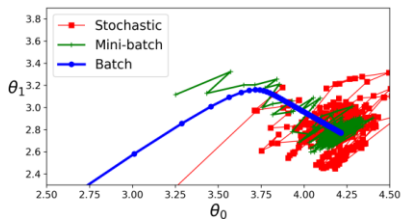

Figure 7: Comparison of batch, mini-batch, and stochastic gradient descent paths in parameter space

4.3 Mini-Batch Gradient Descent

Mini-batch GD strikes a balance by using small batches of data (typically 32, 64, or 128 samples). This approach is widely used in practice, especially for training neural networks.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \frac{1}{b} \sum_{i=1}^{b} \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

Mini-batch gradient with batch size b

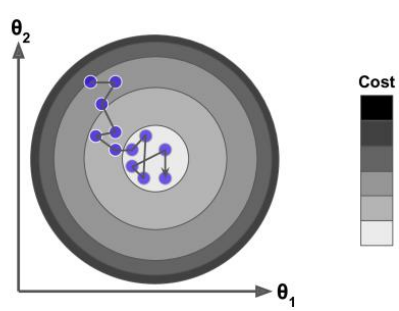

Figure 8: Stochastic gradient descent visualization showing parameter updates across the cost landscape

5. Learning Rate and Convergence

The **learning rate α** is perhaps the most critical hyperparameter in gradient descent. It determines the size of the steps taken towards the minimum:

- Too small α: Slow convergence, risk of getting stuck in local minima
- Too large α: Overshooting, divergence, or oscillation
- Adaptive α: Learning rate schedules decay the rate over time

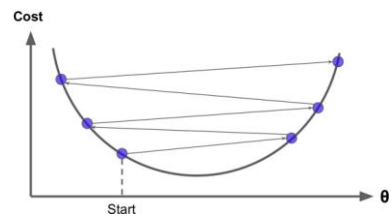

Figure 9: Effect of large learning rate causing divergence or oscillation

5.1 Convergence Criteria

Several criteria can determine when to stop the algorithm:

- **Change in cost**: $|J(\theta_t) - J(\theta_{t-1})| < \varepsilon$
- **Gradient magnitude**: $\|\nabla J(\theta)\| < \varepsilon$
- **Fixed iterations**: Stop after maximum epochs
- **Early stopping**: Based on validation set performance

6. Challenges and Pitfalls

Despite its simplicity, gradient descent faces several challenges in practice:

6.1 Local Minima and Plateaus

Non-convex cost functions (common in neural networks) contain multiple local minima, saddle points, and plateaus. The

algorithm may converge to a suboptimal local minimum or get stuck on a flat plateau.

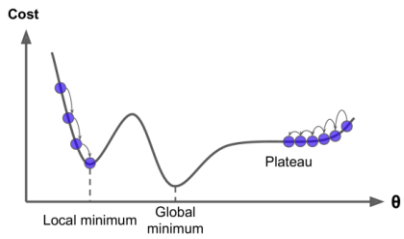

Figure 10: Common pitfalls: local minima, global minimum, and flat plateaus where gradient becomes near-zero

6.2 The Curvature Problem

When features have vastly different scales, the cost function forms elongated elliptical contours. Standard gradient descent zigzags inefficiently rather than taking direct paths.

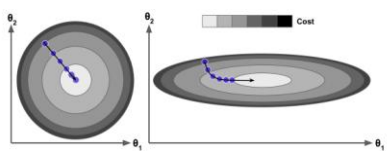

Figure 11: Circular vs. elliptical contours - the latter causes zigzagging and slow convergence

7. Advanced Optimization Methods

To address these challenges, several enhancements to gradient descent have been developed:

Table 2: Advanced optimization algorithms derived from gradient descent

Method	Update Rule	Key Feature
Momentum	$v_t = \beta v_{t-1} + \nabla J(\theta)$; $\theta = \theta - \alpha v_t$	Accumulates velocity in relevant directions
AdaGrad	$\theta = \theta - \alpha/(\sqrt{G} + \varepsilon)$ $\nabla J(\theta)$	Adaptive per-parameter learning rates
RMSprop	$\theta = \theta - \alpha/(\sqrt{E[g^2]} + \varepsilon)\ \nabla J(\theta)$	Exponential moving average of squared gradients
Adam	$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla J(\theta)$; $v_t = \ldots$	Combines momentum and RMSprop benefits

8. Mathematical Analysis

8.1 Convergence for Convex Functions

For convex functions with Lipschitz continuous gradients, gradient descent is guaranteed to converge. If the cost function J is convex and differentiable with L-Lipschitz gradient:

$$\|\nabla J(\mathbf{x}) - \nabla J(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$$

L-Lipschitz condition on the gradient

Then with learning rate $\alpha \leq 1/L$, gradient descent converges with rate $O(1/t)$.

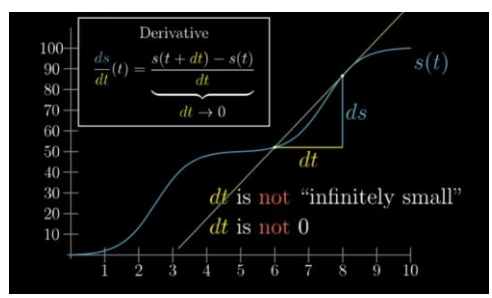

Figure 12: Visualization of a smooth convex function and its convergence properties

8.2 The Hessian and Second-Order Methods

Second-order methods use the **Hessian matrix** (the matrix of second derivatives) for more efficient optimization. Newton's method uses:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - [\nabla^2 J(\boldsymbol{\theta}_t)]^{-1} \nabla J(\boldsymbol{\theta}_t)$$

Newton's method update using the inverse Hessian

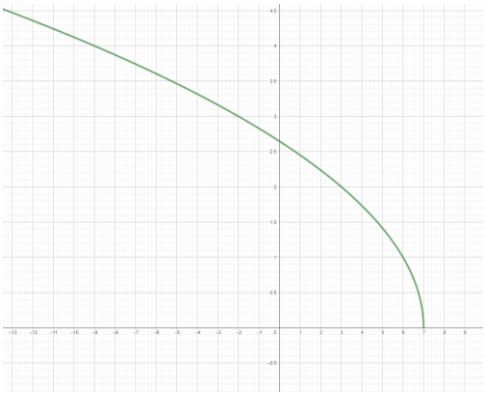

Figure 13: Domain of a function and its geometric representation

9. Practical Considerations

When implementing gradient descent in practice, consider the following guidelines:

Feature Scaling: Always normalize or standardize features to ensure the cost function contours are approximately circular, enabling faster convergence.

Learning Rate Scheduling: Use learning rate decay, step decay, exponential decay, or adaptive schedules to improve convergence.

Initialization: Initialize weights randomly (e.g., Xavier or He initialization) to break symmetry and avoid poor local minima.

Gradient Checking: For debugging, numerically verify that $\nabla J(\theta) \approx (J(\theta + \varepsilon) - J(\theta - \varepsilon))/(2\varepsilon)$ holds.

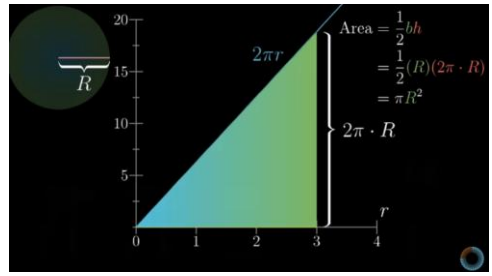

Figure 14: Geometric interpretation of area under curve through integration

10. Summary

Gradient descent remains the cornerstone of modern machine learning optimization. Understanding its mathematical foundations—the gradient, learning rate, convergence properties, and variants—enables practitioners to train models effectively.

From the simple vanilla gradient descent to sophisticated adaptive methods like Adam, these algorithms share the fundamental principle of iteratively descending along the negative gradient to minimize cost functions.

The key to success lies in proper hyperparameter tuning, particularly the learning rate, and understanding the geometry of the optimization landscape through visualization and analysis.

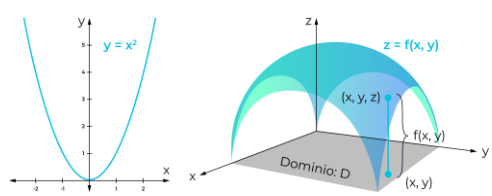

Figure 15: Complex optimization landscape (Himmelblau's function) showing multiple local minima and convergence paths